

Conceito Teóricos: Fundamentos de Hashing e Integridade de Dados (parte 1)

Disciplina: Segurança de sistema de computação

Professor: Ruy de Oliveira

Grupo: Mateus de Souza Arruda e Lorena Strobel Campos

Responsável parte 1: Lorena Strobel (2023278440028)

1.1 Função Hash Criptográfica: O que é uma função hash criptográfica? Cite e detalhe três propriedades essenciais.

A função de hash aceita uma mensagem de tamanho variável como entrada e produz um valor de hash de tamanho fixo. Para aplicações de segurança, utiliza-se uma **função de hash criptográfica**, deve ser um algoritmo que é computacionalmente inviável encontrar um objeto de dados que seja mapeado para um resultado de hash pré-especificado ou encontrar dois objetos de dados que produzam o mesmo valor de hash. Normalmente, a entrada é preenchida até um múltiplo inteiro de algum tamanho fixo e este inclui o valor do tamanho da mensagem original em bits. Dentre suas propriedades essenciais estão: resistência à pré-imagem, resistência à segunda pré-imagem e resistência à colisão.

A **resistência à pré-imagem**, é a propriedade de mão única, ou seja, é fácil gerar um código a partir da mensagem, porém deve ser praticamente impossível obter uma mensagem a partir de seu código. Se a função hash não possuir essa propriedade de mão única, um invasor pode facilmente descobrir o valor secreto, ou seja, tentar recuperar a mensagem original a partir do seu hash. Sendo assim para qualquer valor de hash h informado, é computacionalmente impossível encontrar y , de modo que $H(y) = h$.

A **resistência à segunda pré-imagem** garante que é impossível encontrar uma mensagem diferente com o mesmo valor de hash. Funciona como prevenção contra a falsificação quando for usado um hash encriptado. Se não for verdadeira, um invasor seria capaz de inicialmente observar ou interceptar uma mensagem com seu código de hash encriptado e posteriormente gerar outra mensagem diferente com o mesmo código de hash encriptado. Assim, ela é essencial para garantir que uma mensagem não seja substituída. Logo para qualquer bloco x informado, é computacionalmente impossível encontrar $y \neq x$ com $H(y) = H(x)$.

A **resistência à colisão forte** trata-se de uma propriedade ainda mais forte que as anteriores e que caso a função a possua será chamada de função de hash forte, pois protege

contra um ataque onde uma terceira parte gera uma mensagem para outra parte assinar, isto é, impossível encontrar duas mensagens diferentes que produzem o mesmo valor de hash. Em resumo: é computacionalmente impossível encontrar qualquer par (x, y) , de modo que $H(x) = H(y)$.

1.2 Hash vs. Cifragem: Explique a diferença fundamental entre hash e criptografia (cifragem)

Criptografia consiste nos vários esquemas utilizados para a encriptação transformando textos claros (plaintext) em mensagens codificadas os textos cifrados (ciphertext) utilizando uma chave. Para isso é necessário um algoritmo forte e uma chave com um valor independente do texto claro e do algoritmo. Dessa maneira, a informação que foi cifrada pode posteriormente ser recuperada através do processo de decifração, apanhando o texto cifrado e a chave secreta. Já o hash configura-se como uma mensagem de tamanho variável de produzir um valor de tamanho fixo.

A diferença entre elas consiste justamente no objetivo: a criptografia busca a reversibilidade da mensagem, enquanto o hash deve ser computacionalmente inviável obter a mensagem a partir de seu valor de hash, ou seja o hash deve garantir a confidencialidade da informação, por esse motivo é muito utilizada para determinar se os dados mudaram ou não.

1.3 Colisões e SHA-1: O que é uma colisão? Por que o algoritmo SHA-1 é considerado obsoleto na segurança moderna?

Uma colisão ocorre se tivermos $x \neq y$ e $H(x) = H(y)$, ou seja as entradas são diferentes porém produzem o mesmo valor hash. Como uma das propriedades essenciais é que as funções hash sejam resistentes a colisão, espera-se que seja computacionalmente inviável encontrar duas entradas diferentes.

O SHA-1 é um tipo de função hash, sendo SHA a sigla para Secure Hash Algorithm, ele produz um valor hash de 160 bits que atualmente é considerado obsoleto para aplicações de segurança pois suas resistências a colisões foram quebradas com esforço menor do que o previsto. Desde 2005, o National Institute of Standards and Technology (NIST), agência que desenvolveu o SHA, possuía intenções de retirada da SHA-1, pois, haviam desenvolvido versões com outros tamanho de valor hash. Em 2005, pesquisas descreveram um ataque em que duas mensagens separadas poderiam oferecer o mesmo hash SHA-1 usando menos

operações do que o esperado. Essa situação e diversas outras, como a do ataque denominado SHAttered em 2017 onde dois PDFs distintos possuíam o mesmo SHA-1, aceleraram o processo de transição para o SHA-2 e a NIST estabeleceu que até 2031 algoritmos de padrões antigos sejam abandonados. Por esse motivo o SHA-1 é considerado atualmente como obsoleto para a segurança moderna.

1.4 Mecanismo de Salt: O que é um "salt" e por que ele é indispensável no armazenamento seguro de senhas?

Consiste em adicionar uma sequência de caracteres única e aleatória a cada senha antes de ela ser criptografada, o “salt” é armazenado juntamente com a senha criptografada. Sua principal vantagem é que cada hash é único mesmo que dois usuários tenham a mesma senha. O Salt dificulta ataques pré-computados e impede que o mesmo valor de senha gere o mesmo resultado armazenado. O NIST atualmente recomenda que as senhas armazenadas sejam submetidas a um esquema de derivação de senha com salt.

1.5 Tipos de Ataques a Hashes: Diferencie detalhadamente ataque de força bruta, ataque de dicionário e ataque com rainbow tables.

No ataque de força bruta, o atacante testa todas as chaves possíveis em um trecho do texto cifrado, até obter uma tradução inteligível para o texto claro, em média, metade de todas as chaves possíveis precisa ser experimentada até encontrar a correta. Há mais coisas em um ataque por força bruta do que somente testar chaves é preciso haver um certo grau de conhecimento sobre o texto esperado e algum meio de distinguir automaticamente de dados aleatórios.

O ataque dicionário é fundamentado em dicionários de senhas que já foram previamente construídas, explorando a fraqueza humana de criação de senhas por meio de termos do cotidiano, combinações comuns e outras escolhas previsíveis. Além disso, esses dicionários também podem ser obtidos a partir de senhas que foram vazadas de diversas organizações ao longo do tempo. Dessa forma, o ataque dicionário mostra-se mais eficiente do que o ataque por força bruta, pois o atacante testa primeiro as possibilidades com maior probabilidade de serem utilizadas.

O ataque de Rainbow tables utiliza tabelas pré-computadas especiais para quebrar os hashes das senhas em um banco de dados. Essas tabelas são construídas previamente e

armazenam informações que permitem relacionar possíveis textos simples a seus respectivos hashes. Dessa maneira, o atacante pode comparar um hash obtido de um banco de dados com os valores presentes ou derivados da tabela.

REFERÊNCIAS:

GOOGLE. **SHattered: The first collision for SHA-1**. Disponível em:

<https://shattered.io/pt/colisao-sha1/>. Acesso em: 4 set. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **NIST Retires SHA-1 Cryptographic Algorithm**. 2022. Disponível em:

<https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm>. Acesso em: 4 set. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **SP 800-63B: Digital Identity Guidelines**. Disponível em: <https://pages.nist.gov/800-63-4/sp800-63b.html>. Acesso em: 4 set. 2026.

RAMOS, Marlon Brendo. **Redes Neurais Recorrentes para Geração de Senhas em Ataques de Força Bruta baseado em Dicionário**. 2022. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) — Universidade Federal de Uberlândia, Monte Carmelo, 2022. Disponível em: <https://repositorio.ufu.br/bitstream/123456789/36723/1/RedesNeuraisRecorrentes.pdf>. Acesso em: 4 set. 2026.